

The Conjugate Gradient Method

From Quadratic Minimization to Nonlinear Optimization

ISE 5405/5406 Tutorial

Abstract

This tutorial develops the conjugate gradient (CG) method from first principles. We begin with the problem of minimizing a convex quadratic, where CG can be understood as both a direct method and an iterative method with a clean convergence theory rooted in Krylov subspaces and Chebyshev polynomials. We then extend to the nonlinear setting, where several variants of the β update formula lead to methods with different practical and theoretical properties. Throughout, we compare CG to steepest descent and quasi-Newton methods to clarify when each is the right tool.

Contents

1	Motivation: What's Wrong with Steepest Descent?	2
2	The Linear Conjugate Gradient Method	3
2.1	Conjugate Directions	3
2.2	The CG Algorithm	4
2.3	Fundamental Properties	5
3	Convergence Analysis	5
3.1	The Chebyshev Bound	5
3.2	Superlinear Convergence and Eigenvalue Clustering	6
4	Nonlinear Conjugate Gradient	7
4.1	General Framework	7
4.2	The β Formulas	7
4.3	Practical Differences	8
4.4	Restarts	8
5	CG in the Optimization Landscape	9
5.1	Comparison with Other Methods	9
5.2	Linear CG as a Linear Solver	9
5.3	CG for Newton Systems	10
6	Summary of Key Ideas	10

1 Motivation: What's Wrong with Steepest Descent?

Consider the unconstrained minimization problem

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{2}x^T A x - b^T x, \quad (1)$$

where $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite. The unique minimizer is $x^* = A^{-1}b$, and the gradient is $\nabla f(x) = Ax - b$. We write $g_k = \nabla f(x_k) = Ax_k - b$ for the gradient (equivalently, the negative residual $g_k = -r_k$ where $r_k = b - Ax_k$).

Steepest descent uses the iteration

$$x_{k+1} = x_k - \alpha_k g_k, \quad \alpha_k = \frac{g_k^T g_k}{g_k^T A g_k},$$

where α_k is the exact minimizer along the steepest descent direction $-g_k$.

This method has a well-known pathology: on ill-conditioned problems, it *zigzags*. Successive gradients are orthogonal (a consequence of the exact line search), but orthogonality in the Euclidean inner product does not mean the search directions efficiently explore the space in the geometry defined by A .

Steepest Descent vs. Conjugate Gradient on $f(x) = \frac{1}{2}x^T A x$

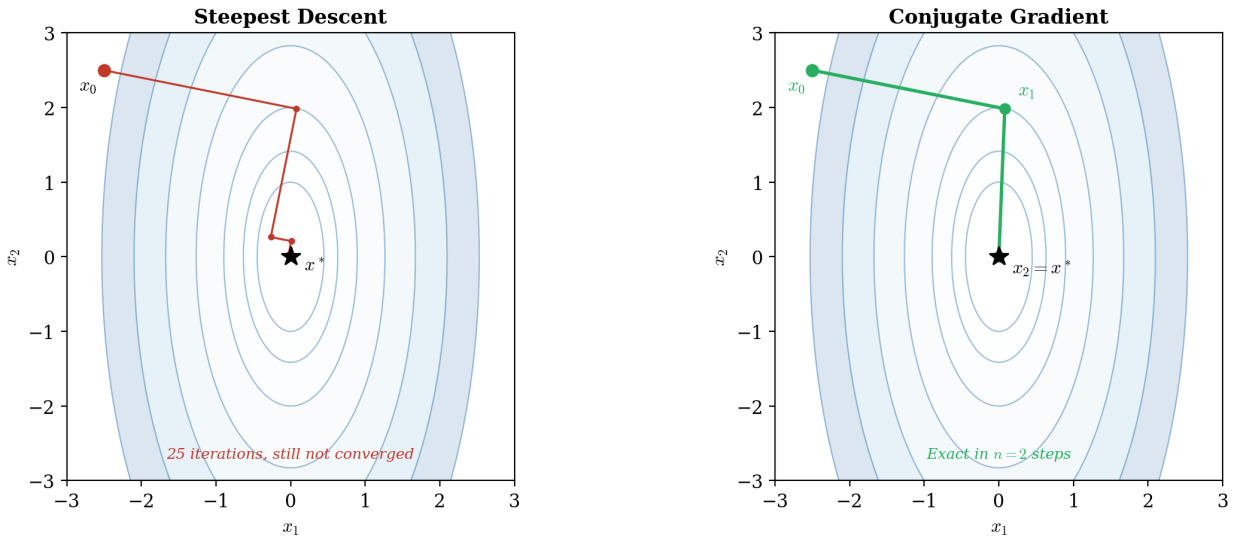


Figure 1: Steepest descent vs. conjugate gradient on a quadratic with $A = \text{diag}(5, 1)$, $\kappa(A) = 5$. Starting from the same point, SD takes 25+ iterations and zigzags along the narrow valley, while CG converges in exactly $n = 2$ steps.

The convergence rate of steepest descent on (1) is governed by the condition number $\kappa = \lambda_{\max}(A)/\lambda_{\min}(A)$:

$$\|x_{k+1} - x^*\|_A \leq \frac{\kappa - 1}{\kappa + 1} \|x_k - x^*\|_A, \quad (2)$$

where $\|v\|_A = \sqrt{v^T A v}$ is the A -norm (also called the energy norm). For $\kappa = 100$, this gives a convergence factor of $\rho \approx 0.98$ —meaning each iteration reduces the error by only about 2%. The question is: can we do better without resorting to second-order information?

2 The Linear Conjugate Gradient Method

2.1 Conjugate Directions

The key idea is to replace Euclidean orthogonality with A -conjugacy.

Definition 2.1 (A -conjugacy). Nonzero vectors $d_0, d_1, \dots, d_m$ are A -conjugate (or simply *conjugate* when A is clear from context) if

$$d_i^T A d_j = 0 \quad \text{for all } i \neq j.$$

A set of A -conjugate vectors is linearly independent (since A is positive definite, $\langle \cdot, \cdot \rangle_A = (\cdot)^T A (\cdot)$ defines an inner product, and conjugacy is orthogonality in this inner product). Consequently, at most n nonzero A -conjugate directions can exist in $\mathbb{R}^n$.

The geometric interpretation is shown in Figure 2: directions that appear non-orthogonal in the original coordinate system become genuinely perpendicular after the change of variables $y = A^{1/2}x$ that transforms the elliptical contours of f into circles.

A -Conjugate Directions: $d_i^T A d_j = 0$

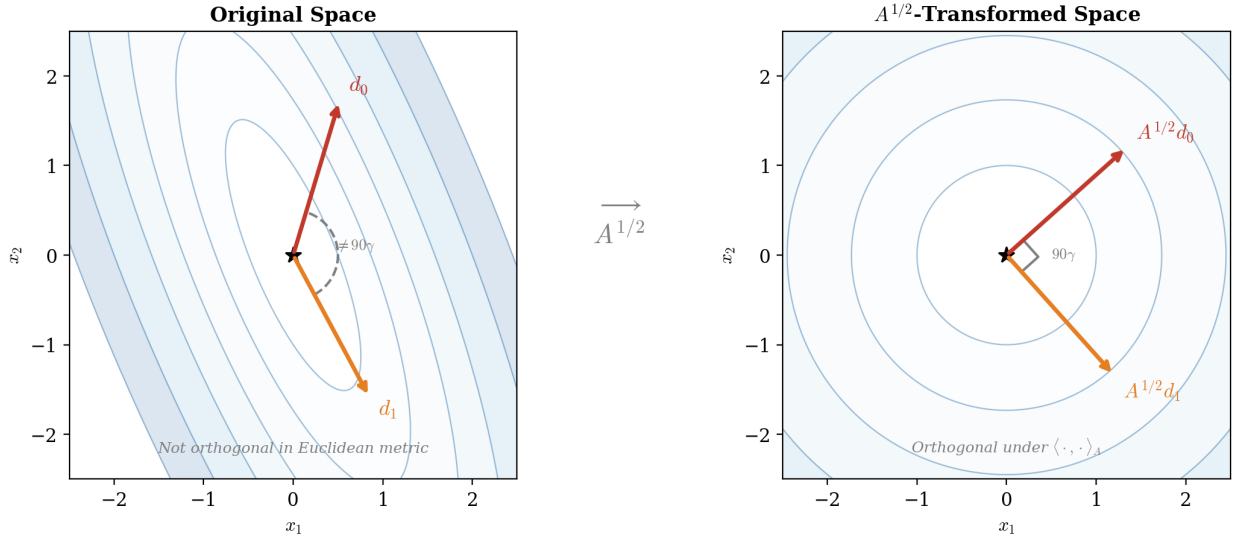


Figure 2: A -conjugate directions in the original space (left) and after the $A^{1/2}$ transformation (right). The directions are not Euclidean-orthogonal, but they are orthogonal in the A -inner product, which corresponds to ordinary orthogonality in the transformed space where the contours are circular.

Proposition 2.2 (Conjugate direction method). *Given n A -conjugate directions $d_0, \dots, d_{n-1}$ and any starting point x_0 , the iteration*

$$x_{k+1} = x_k + \alpha_k d_k, \quad \alpha_k = -\frac{g_k^T d_k}{d_k^T A d_k},$$

converges to $x^ = A^{-1}b$ in at most n steps.*

Proof. Since $\{d_0, \dots, d_{n-1}\}$ forms a basis for $\mathbb{R}^n$, we can write $x^* - x_0 = \sum_{i=0}^{n-1} \sigma_i d_i$ for unique coefficients σ_i . Premultiplying by $d_j^T A$ and using A -conjugacy:

$$d_j^T A(x^* - x_0) = \sigma_j (d_j^T A d_j).$$

Since $A(x^* - x_0) = b - Ax_0 = -g_0$, we obtain $\sigma_j = -d_j^T g_0 / (d_j^T A d_j)$ for every j . It remains to show $\alpha_k = \sigma_k$. We have $g_k = g_0 + A \sum_{i=0}^{k-1} \alpha_i d_i$, so $d_k^T g_k = d_k^T g_0 + \sum_{i < k} \alpha_i d_k^T A d_i = d_k^T g_0$ by conjugacy. Therefore $\alpha_k = -g_k^T d_k / (d_k^T A d_k) = -d_k^T g_0 / (d_k^T A d_k) = \sigma_k$. After n steps, all components of $x^* - x_0$ in the conjugate basis have been recovered. $\square$

2.2 The CG Algorithm

The conjugate gradient method constructs the A -conjugate directions *on the fly* using only matrix–vector products with A :

Algorithm 1 Linear Conjugate Gradient (Hestenes–Stiefel, 1952)

```

1: Input:  $A \succ 0$ ,  $b$ , starting point  $x_0$ 
2:  $g_0 \leftarrow Ax_0 - b$ ,  $d_0 \leftarrow -g_0$ 
3: for  $k = 0, 1, 2, \dots$  do
4:    $\alpha_k \leftarrow \frac{g_k^T g_k}{d_k^T A d_k}$  ▷ Step length (exact line search)
5:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$  ▷ Update iterate
6:    $g_{k+1} \leftarrow g_k + \alpha_k A d_k$  ▷ Update gradient
7:   if  $\|g_{k+1}\| < \varepsilon$  then stop
8:   end if
9:    $\beta_{k+1} \leftarrow \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}$  ▷ CG coefficient
10:   $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$  ▷ New conjugate direction
11: end for

```

The cost per iteration is dominated by *one* matrix–vector product Ad_k (used in both lines 4 and 6). Storage is $O(n)$: only the vectors x_k , g_k , d_k , and the product Ad_k need to be held simultaneously.

Remark 2.3 (Step length simplification). The general conjugate direction step length (Proposition 2.2) is $\alpha_k = -g_k^T d_k / (d_k^T A d_k)$. In Algorithm 1, this simplifies to $\alpha_k = g_k^T g_k / (d_k^T A d_k)$ because $g_k^T d_k = -\|g_k\|^2$. To see this: $d_k = -g_k + \beta_k d_{k-1}$ gives $g_k^T d_k = -\|g_k\|^2 + \beta_k g_k^T d_{k-1}$, and $g_k^T d_{k-1} = 0$ by the mutual orthogonality of gradients (Theorem 2.4(a), verified inductively).

2.3 Fundamental Properties

The following properties hold in exact arithmetic and are the theoretical backbone of CG.

Theorem 2.4 (CG Invariants). *After k steps of Algorithm 1:*

- (a) **Mutual orthogonality of gradients:** $g_i^T g_j = 0$ for $i \neq j$.
- (b) **A-conjugacy of directions:** $d_i^T A d_j = 0$ for $i \neq j$.
- (c) **Krylov subspace property:**

$$\text{span}\{g_0, g_1, \dots, g_k\} = \text{span}\{d_0, d_1, \dots, d_k\} = \mathcal{K}_{k+1}(A, g_0),$$

where $\mathcal{K}_{k+1}(A, g_0) = \text{span}\{g_0, A g_0, A^2 g_0, \dots, A^k g_0\}$ is the $(k+1)$ -th Krylov subspace.

- (d) **Optimality over Krylov subspace:**

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k(A, g_0)} f(x).$$

Property (d) is the most important: CG finds the *best possible* iterate within the subspace that can be built from gradient information and powers of A . No method using only matrix–vector products with A can do better.

Corollary 2.5 (Finite termination). *Since $\dim \mathcal{K}_k \leq n$ and $\mathcal{K}_k \subseteq \mathcal{K}_{k+1}$ with strict inclusion until $\mathcal{K}_k = \mathbb{R}^n$, CG terminates in at most n steps.*

3 Convergence Analysis

3.1 The Chebyshev Bound

The Krylov optimality property (Theorem 2.4(d)) leads directly to a convergence bound via polynomial approximation.

Theorem 3.1 (CG convergence bound). *Let $0 < \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ be the eigenvalues of A . Then*

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq \min_{\substack{p \in \mathcal{P}_k \\ p(0)=1}} \max_{1 \leq i \leq n} |p(\lambda_i)|, \quad (3)$$

where $\mathcal{P}_k$ denotes the set of polynomials of degree at most k .

The minimization on the right-hand side is a classical Chebyshev approximation problem. Using the Chebyshev polynomial of the first kind T_k on the interval $[\lambda_1, \lambda_n]$ gives:

Corollary 3.2 (Condition number bound).

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k, \quad (4)$$

where $\kappa = \lambda_n / \lambda_1$ is the spectral condition number of A .

Comparing (4) with the steepest descent bound (2), CG effectively replaces κ with $\sqrt{\kappa}$ in the convergence factor. For $\kappa = 100$:

$$\rho_{\text{SD}} = \frac{99}{101} \approx 0.980, \quad \rho_{\text{CG}} = \frac{9}{11} \approx 0.818.$$

To reach a tolerance of 10^{-6} , SD needs roughly $\frac{6 \ln 10}{\ln(101/99)} \approx 697$ iterations, while CG needs roughly $\frac{\ln(2 \times 10^6)}{\ln(11/9)} \approx 72$ (accounting for the leading constant of 2). The improvement is dramatic, and grows with κ .

Convergence Rate: Steepest Descent vs. Conjugate Gradient

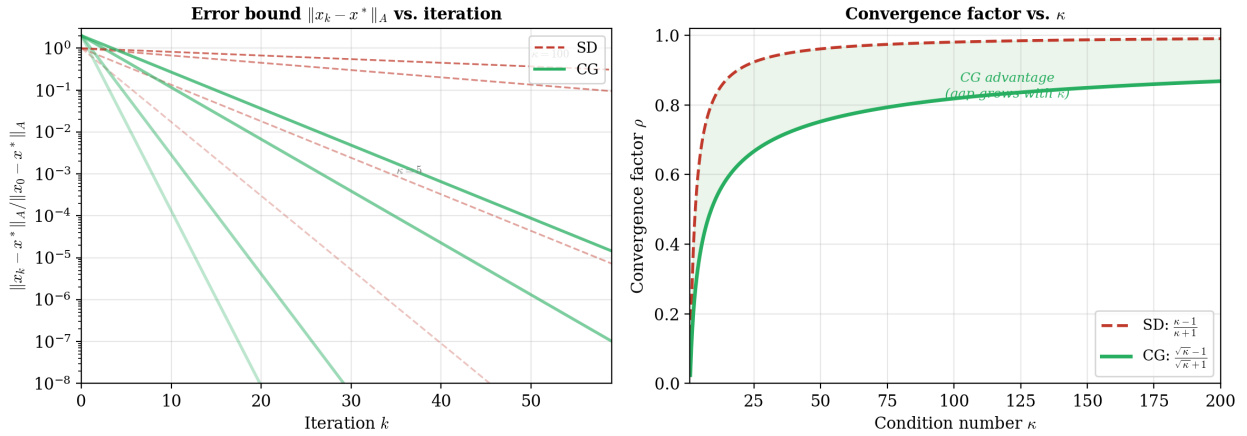


Figure 3: Left: error decay for SD (dashed) and CG (solid) across several condition numbers. Right: convergence factor ρ as a function of κ . CG effectively square-roots the condition number, and the gap grows with κ .

3.2 Superlinear Convergence and Eigenvalue Clustering

The bound (4) is often pessimistic. The polynomial approximation in (3) depends on the *distribution* of eigenvalues, not just the extremes. Specifically:

- If A has only m distinct eigenvalues, CG terminates in at most $m \leq n$ steps.
- If the eigenvalues cluster into m groups, CG converges as though the problem has dimension m , not n .
- Outlier eigenvalues are “captured” early: after the first few iterations, CG behaves as though those eigenvalues do not exist.

This eigenvalue sensitivity is the theoretical basis for *preconditioning*: given a preconditioner $M \approx A$, one applies CG to the system $M^{-1}Ax = M^{-1}b$. Although $M^{-1}A$ is not symmetric in the Euclidean inner product, it is self-adjoint in the M -inner product $\langle u, v \rangle_M = u^T M v$, and the preconditioned CG (PCG) algorithm is formulated to exploit this structure. If M clusters the eigenvalues of $M^{-1}A$ near 1, CG on the preconditioned system converges rapidly regardless of the original condition number. PCG maintains the same structure as Algorithm 1 with an additional preconditioning solve $Mz_{k+1} = g_{k+1}$ per iteration.

4 Nonlinear Conjugate Gradient

For general (nonquadratic) unconstrained minimization $\min_{x \in \mathbb{R}^n} f(x)$, we no longer have an explicit matrix A , so the exact line search formula and the conjugacy property from the linear case do not carry over directly. The idea of nonlinear CG is to *mimic* the structure of Algorithm 1 using only gradient evaluations.

4.1 General Framework

Algorithm 2 Nonlinear Conjugate Gradient

```

1: Input: Starting point  $x_0$ , tolerance  $\varepsilon > 0$ 
2:  $g_0 \leftarrow \nabla f(x_0)$ ,  $d_0 \leftarrow -g_0$ 
3: for  $k = 0, 1, 2, \dots$  do
4:   Find  $\alpha_k > 0$  satisfying Wolfe conditions ▷ Line search
5:    $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
6:    $g_{k+1} \leftarrow \nabla f(x_{k+1})$ 
7:   if  $\|g_{k+1}\| < \varepsilon$  then stop
8:   end if
9:   Compute  $\beta_{k+1}$  ▷ Choice of formula—see below
10:   $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$ 
11: end for

```

The critical design choices are:

- (i) The **line search**: typically strong Wolfe conditions with $c_1 = 10^{-4}$, $c_2 = 0.1$.
- (ii) The β **formula**: this is where the variants diverge.
- (iii) A **restart strategy**: periodically resetting $d_k = -g_k$.

4.2 The β Formulas

On a quadratic with exact line search, all of the following formulas are equivalent. For general nonlinear functions, they produce different sequences and have different convergence properties.

Let $y_k = g_{k+1} - g_k$ denote the gradient difference. The main formulas are:

Name	Formula	Reference
Fletcher–Reeves (FR)	$\beta_{k+1}^{\text{FR}} = \frac{\ g_{k+1}\ ^2}{\ g_k\ ^2}$	Fletcher & Reeves, 1964
Polak–Ribière (PR)	$\beta_{k+1}^{\text{PR}} = \frac{g_{k+1}^T y_k}{\ g_k\ ^2}$	Polak & Ribière, 1969
Polak–Ribière+ (PR+)	$\beta_{k+1}^{\text{PR+}} = \max(0, \beta_{k+1}^{\text{PR}})$	Gilbert & Nocedal, 1992
Hestenes–Stiefel (HS)	$\beta_{k+1}^{\text{HS}} = \frac{g_{k+1}^T y_k}{d_k^T y_k}$	Hestenes & Stiefel, 1952
Dai–Yuan (DY)	$\beta_{k+1}^{\text{DY}} = \frac{\ g_{k+1}\ ^2}{d_k^T y_k}$	Dai & Yuan, 1999

4.3 Practical Differences

Fletcher–Reeves. Has a clean global convergence proof (Al-Baali, 1985; Gilbert & Nocedal, 1992) under strong Wolfe line search. However, β^{FR} can remain large even when the method is making poor progress, leading to search directions far from the steepest descent direction. This manifests as slow convergence on difficult problems.

Polak–Ribière and PR+. The key advantage is the numerator $g_{k+1}^T(g_{k+1} - g_k)$: when successive gradients are nearly parallel (as happens on flat regions or near convergence), the numerator is small, so $\beta \approx 0$ and the method *automatically restarts* toward steepest descent. This self-correcting behavior makes PR much more robust in practice. The “+” variant (truncation at zero) restores global convergence, which the original PR formula lacks.

Dai–Yuan. Has the theoretical advantage of guaranteed global convergence under the standard Wolfe line search, without requiring restarts. However, its practical performance is often between FR and PR+.

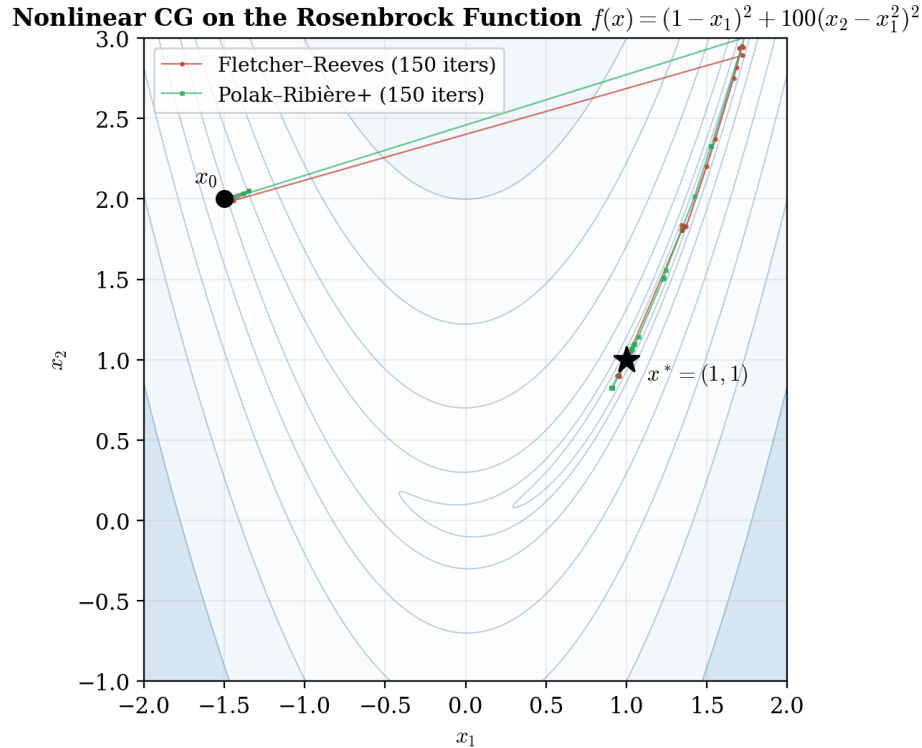


Figure 4: Fletcher–Reeves vs. Polak–Ribière+ on the Rosenbrock function $f(x) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$. Both start at $(-1.5, 2)$ and use backtracking Armijo line search. PR+ typically reaches the minimizer $(1, 1)$ in fewer iterations due to its automatic restart behavior.

4.4 Restarts

Even with a good β formula, the conjugate directions degrade over time on nonquadratic problems. Practical implementations restart the method (setting $d_k \leftarrow -g_k$) periodically. Common strategies include:

- **Fixed restart** every n iterations (or every $\sqrt{n}$ iterations for very large n).
- **Powell restart**: restart whenever $|g_{k+1}^T g_k| / \|g_{k+1}\|^2 > \nu$ for some threshold ν (e.g., $\nu = 0.1$). This detects loss of orthogonality between successive gradients.
- **Descent check**: restart if d_k is not a sufficient descent direction, i.e., $g_k^T d_k > -\delta \|g_k\| \|d_k\|$ for a small $\delta > 0$.

5 CG in the Optimization Landscape

5.1 Comparison with Other Methods

Method	Storage	Cost/iter	Conv. rate	Best for
Steepest descent	$O(n)$	∇f	Linear, $\rho \sim \frac{\kappa-1}{\kappa+1}$	Prototyping
Nonlinear CG	$O(n)$	∇f	Linear*	Very large n
L-BFGS	$O(mn)$	∇f	Superlinear	Large n , $m \ll n$
BFGS	$O(n^2)$	∇f	Superlinear	Moderate n
Newton (modified)	$O(n^2)$	$\nabla f, \nabla^2 f$	Quadratic	Small–moderate n

*For linear CG on quadratics, the rate involves $\sqrt{\kappa}$; for nonlinear CG, convergence is typically linear but faster than SD.

The practical hierarchy for most problems is:

1. If n is moderate and $\nabla^2 f$ is available $\rightarrow$ modified Newton.
2. If n is moderate and only ∇f is available $\rightarrow$ BFGS.
3. If n is large but $O(mn)$ storage is feasible $\rightarrow$ L-BFGS.
4. If n is truly enormous or memory is extremely tight $\rightarrow$ nonlinear CG.

L-BFGS usually dominates nonlinear CG in wall-clock time because the accumulated curvature information (stored in m vector pairs, typically $m = 5\text{--}20$) produces better search directions. The cases where CG wins are those where even $O(mn)$ storage is infeasible, which arises in some PDE-constrained and machine learning problems.

5.2 Linear CG as a Linear Solver

In scientific computing, the most common use of CG is not as an optimizer but as an iterative solver for $Ax = b$ with $A \succ 0$. This is the same algorithm (since solving $Ax = b$ is equivalent to minimizing (1)), but the perspective shifts:

- A is typically large, sparse, and arises from a PDE discretization.
- A is never formed explicitly; only the action $v \mapsto Av$ is needed.

- **Preconditioning** is essential: the preconditioned CG (PCG) algorithm solves $M^{-1}Ax = M^{-1}b$ where $M \approx A$ is easy to invert. Good preconditioners (incomplete Cholesky, algebraic multigrid, domain decomposition) can reduce the effective condition number from $O(h^{-2})$ to $O(1)$, where h is the mesh size.
- The convergence theory based on eigenvalue clustering (Section 3.2) explains why preconditioned CG often converges in far fewer than n iterations, even for n in the millions.

5.3 CG for Newton Systems

An important bridge between linear CG and nonlinear optimization is the *Newton-CG* method (also called the truncated Newton method). At each outer iteration, instead of solving the Newton system $\nabla^2 f(x_k) d = -g_k$ exactly, we run a few steps of linear CG to get an approximate Newton direction. This gives superlinear convergence near the solution (and quadratic convergence if the inner CG solve is sufficiently accurate, cf. Dembo–Eisenstat–Steihaug), with a cost of $O(n)$ per CG step (since we only need Hessian–vector products $\nabla^2 f \cdot v$, which can be computed by automatic differentiation or finite differences without forming $\nabla^2 f$ explicitly).

6 Summary of Key Ideas

1. **Conjugacy replaces orthogonality.** By choosing search directions that are orthogonal in the A -inner product rather than the Euclidean inner product, CG avoids the zigzagging of steepest descent and converges in at most n steps on a quadratic.
2. **Krylov optimality.** At each step, CG produces the best possible iterate in the Krylov subspace $x_0 + \mathcal{K}_k(A, g_0)$. This is the optimal use of k matrix–vector products.
3. **The $\sqrt{\kappa}$ effect.** CG’s convergence factor involves $\sqrt{\kappa}$ where steepest descent has κ . This is a consequence of Chebyshev polynomial approximation and is the key theoretical advantage.
4. **Eigenvalue sensitivity.** CG converges fast when eigenvalues are clustered, not just when the condition number is small. This is why preconditioning is so effective.
5. **Nonlinear extensions.** Several β formulas mimic the linear CG structure for general f . Polak–Ribière+ is usually the best practical choice due to its automatic restart behavior.
6. **$O(n)$ storage.** CG’s minimal memory footprint makes it the method of last resort when L-BFGS is too expensive, and the method of first resort as a linear solver inside Newton-type methods.

References

- Hestenes, M.R. and Stiefel, E. (1952). Methods of conjugate gradients for solving linear systems. *J. Research Nat. Bur. Standards*, 49(6):409–436.
- Fletcher, R. and Reeves, C.M. (1964). Function minimization by conjugate gradients. *The Computer Journal*, 7(2):149–154.
- Polak, E. and Ribière, G. (1969). Note sur la convergence de méthodes de directions conjuguées. *Revue Française d’Informatique et de Recherche Opérationnelle*, 3(16):35–43.

Gilbert, J.C. and Nocedal, J. (1992). Global convergence properties of conjugate gradient methods for optimization. *SIAM J. Optimization*, 2(1):21–42.

Dai, Y.H. and Yuan, Y. (1999). A nonlinear conjugate gradient method with a strong global convergence property. *SIAM J. Optimization*, 10(1):177–182.

Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization*, 2nd ed., Springer. Chapters 5 and 7.

Shewchuk, J.R. (1994). An introduction to the conjugate gradient method without the agonizing pain. Technical report, CMU. (An excellent and widely-cited tutorial.)